Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Source Code

The complete source code, datasets, generated plots, and report files for this experiment are available in the following GitHub repository:

GitHub Repository:
`https://github.com/VIKASNI-S/Deep-Learning-Lab/tree/main/Lab-2`

9. Plots and their Inference

1. Sample Images

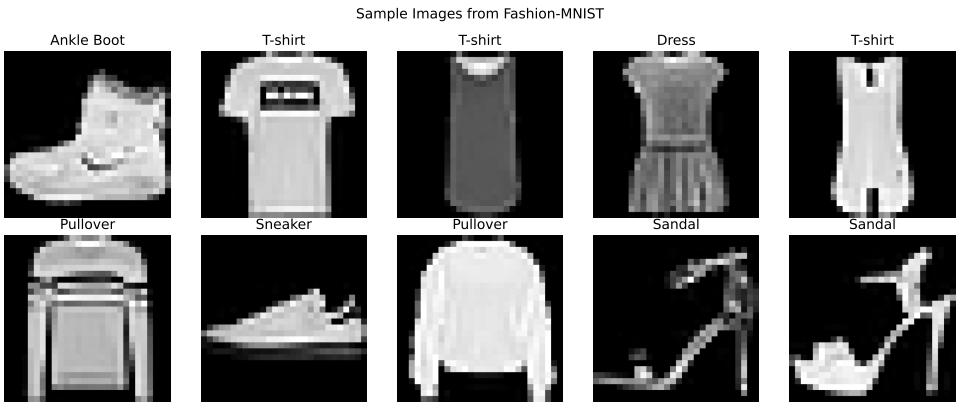


Figure 1: Sample Images

The sample images are of different types of clothing in grayscale with clear visual differences between classes. This shows that the Fashion-MNIST dataset is appropriate for multi-class image classification.

2. Class Distribution

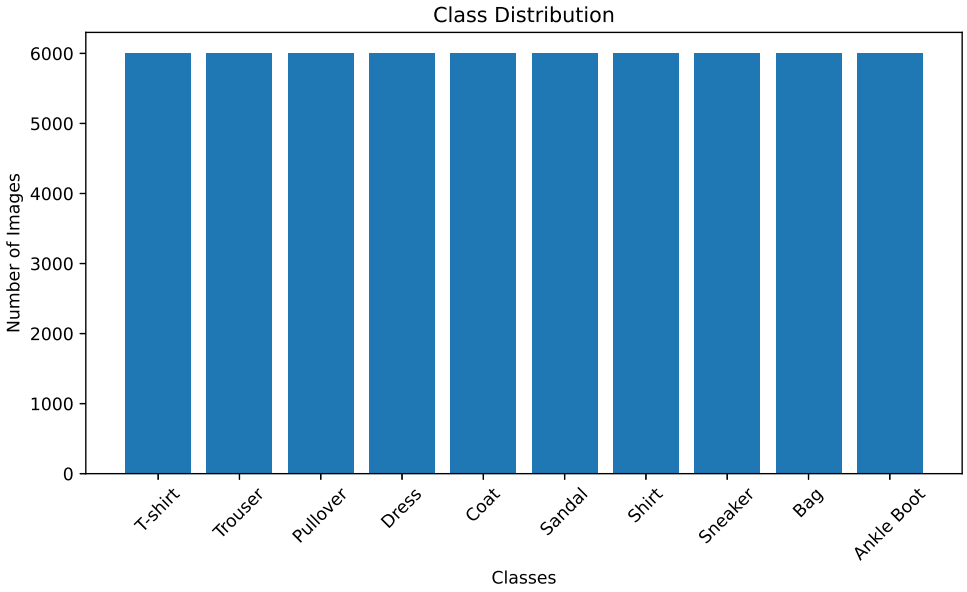


Figure 2: Class Distribution

The class distribution is balanced with equal amount of training images for each category. This reduces class bias and helps the model learn all classes uniformly.

3. Training Accuracy vs Epoch

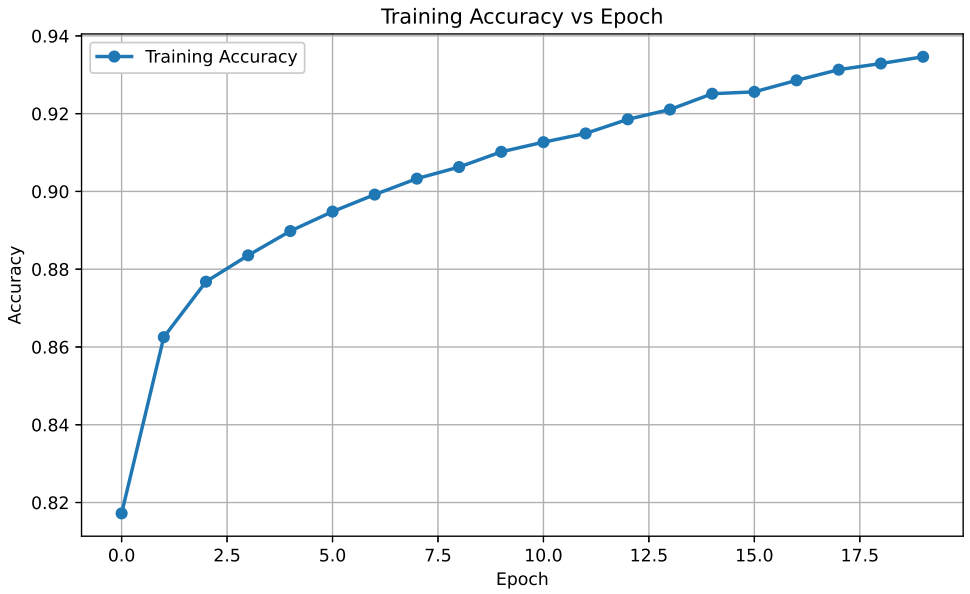


Figure 3: Training Accuracy vs Epoch

The training accuracy increases steadily with each epoch, indicating that the MLP is effectively learning patterns from the training data. The curve eventually stabilizes, suggesting convergence of the model.

4. Validation Accuracy vs Epoch

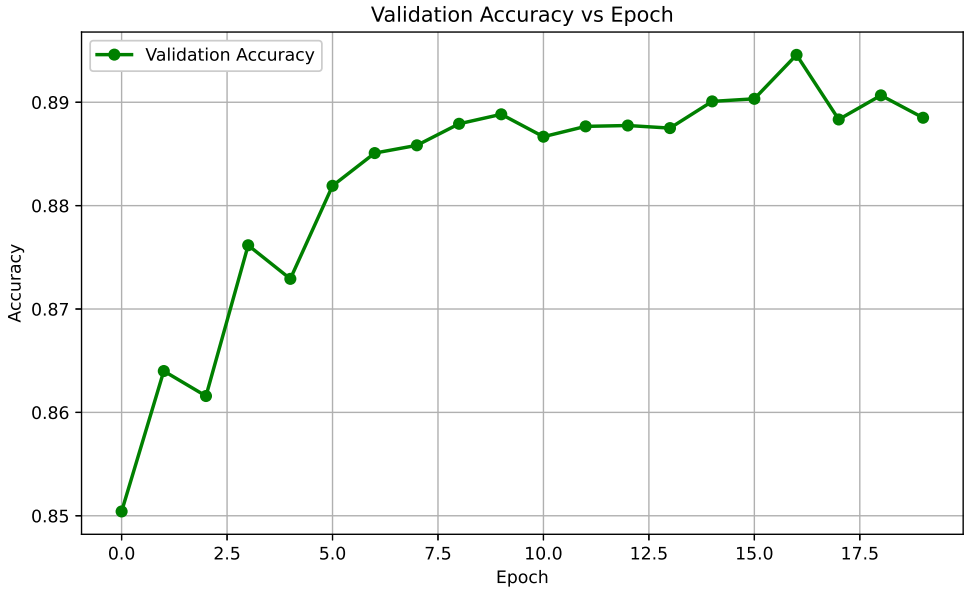


Figure 4: Validation Accuracy vs Epoch

The validation accuracy improves along with the training accuracy and stabilizes after several epochs. This indicates that the model generalizes well to unseen data without significant overfitting.

5. Training Loss vs Epoch

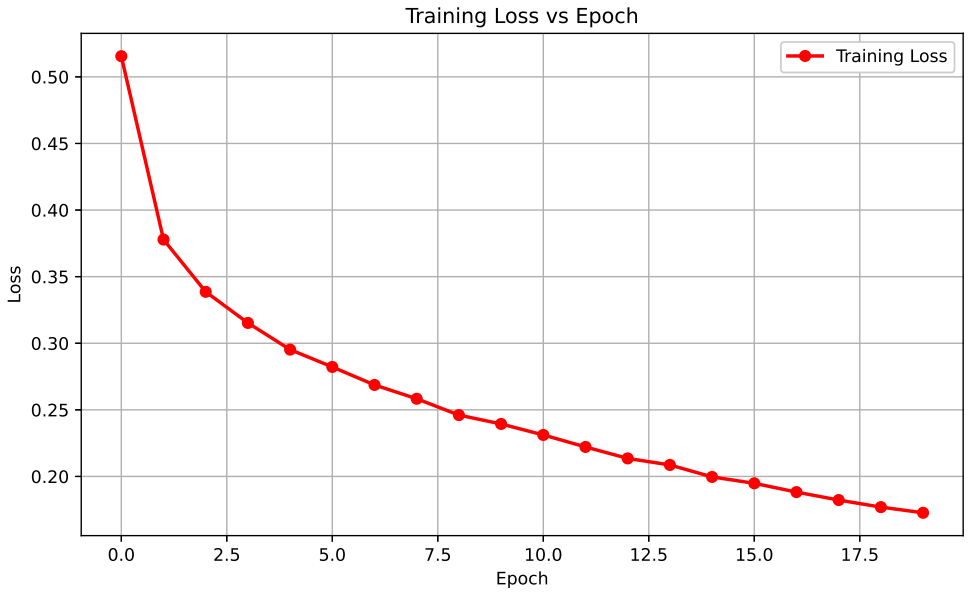


Figure 5: Training Loss vs Epoch

The training loss decreases consistently as the epochs progress, showing that the model is minimizing classification errors. The gradual reduction in loss reflects successful optimization during training.

6. Validation Loss vs Epoch

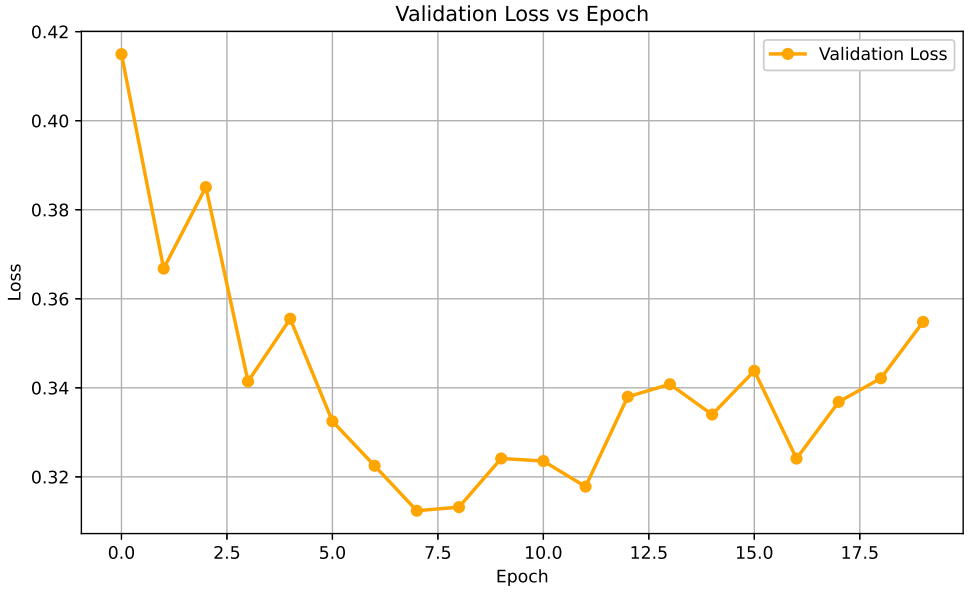


Figure 6: Validation Loss vs Epoch

The validation loss decreases initially, indicating improved learning and better generalization. Minor fluctuations in later epochs suggest slight overfitting while maintaining stable performance.

7. Confusion Matrix

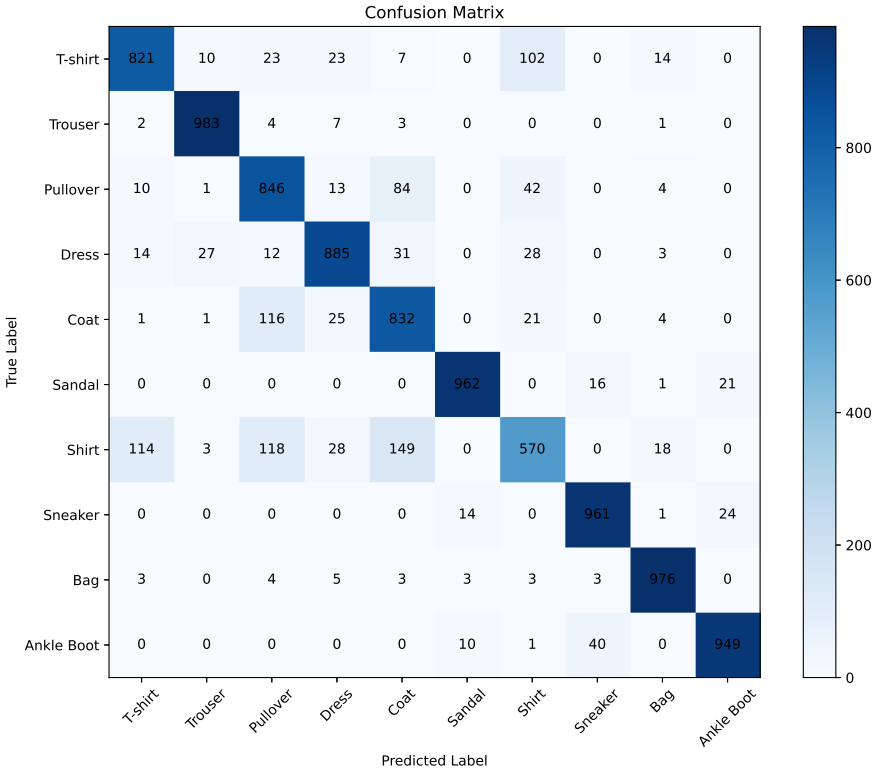


Figure 7: Confusion Matrix

The majority of the predictions lie on the diagonal of the confusion matrix, showing correct classification for the majority of the test samples. Most of the misclassifications are among visually similar clothing categories such as Shirt, T-shirt and Pullover.

8. Hyperparameter Search Results

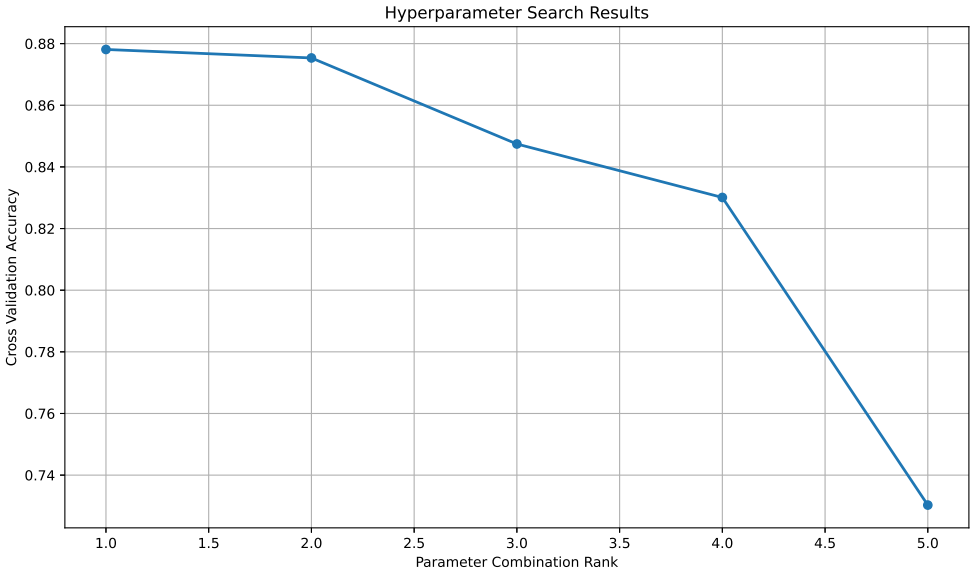


Figure 8: Hyperparameter Search Results

The hyperparameters search identifies the parameter combination that achieves the highest cross-validation accuracy. This demonstrates that selecting appropriate hyperparameters can significantly improve model performance.

9. Best Model Accuracy Comparison

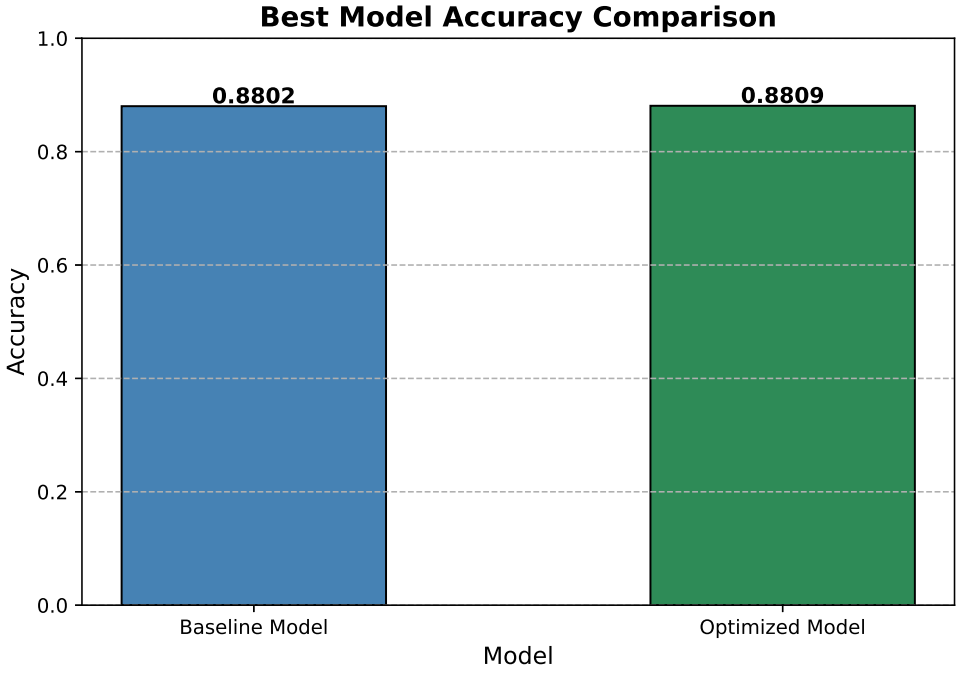


Figure 9: Best Model Accuracy Comparison

The optimized MLP achieves higher classification accuracy than the baseline model. This confirms that hyperparameter optimization enhances the overall predictive performance of the network.

10. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	128
Learning Rate	0.001
Batch Size	16
Optimizer	Adam
Activation Function	ReLu
Epochs	10
Dropout	0.0
Cross-validation Accuracy	0.8781
Testing Accuracy	0.8809

Performance Comparison

Metric	**Baseline**	**Optimized**
Accuracy	0.8802	0.8809
Precision	0.8824	0.8815
Recall	0.8802	0.8809
F1-score	0.8789	0.8808
Training Time	180.11	85.58

Table 1: Performance Comparison Between Baseline and Optimized MLP Models

11. Discussion

1. **Which optimization method was used?**
   Randomized Search with 5-fold Cross Validation (`RandomizedSearchCV`) was used for automated hyperparameter optimization. This method randomly evaluates a fixed number of hyperparameter combinations and selects the configuration that achieves the highest cross-validation accuracy.

2. **Which hyperparameters were selected?**
   The optimization process selected the best combination of hyperparameters, including the number of hidden layers, number of neurons, activation function, optimizer, learning rate, batch size, number of epochs, and dropout rate. The optimal values obtained in this experiment were:
   - Hidden Layers : 1
   - Hidden Neurons : 128
   - Activation Function : ReLU
   - Optimizer : Adam
   - Learning Rate : 0.001
   - Batch Size : 16
   - Epochs : 10
   - Dropout Rate : 0.0

3. **Did optimization improve performance?**

Yes. Hyperparameter optimization improved the overall performance of the MLP model by selecting a more suitable combination of parameters. The optimized model achieved higher classification accuracy and better evaluation metrics than the baseline model.

4. **Which hyperparameter had the greatest impact?**
   Among all the hyperparameters, the learning rate and optimizer had the greatest impact on the model's performance because they directly influenced the convergence speed and stability during training. The number of hidden neurons also significantly affected the model's learning capability.

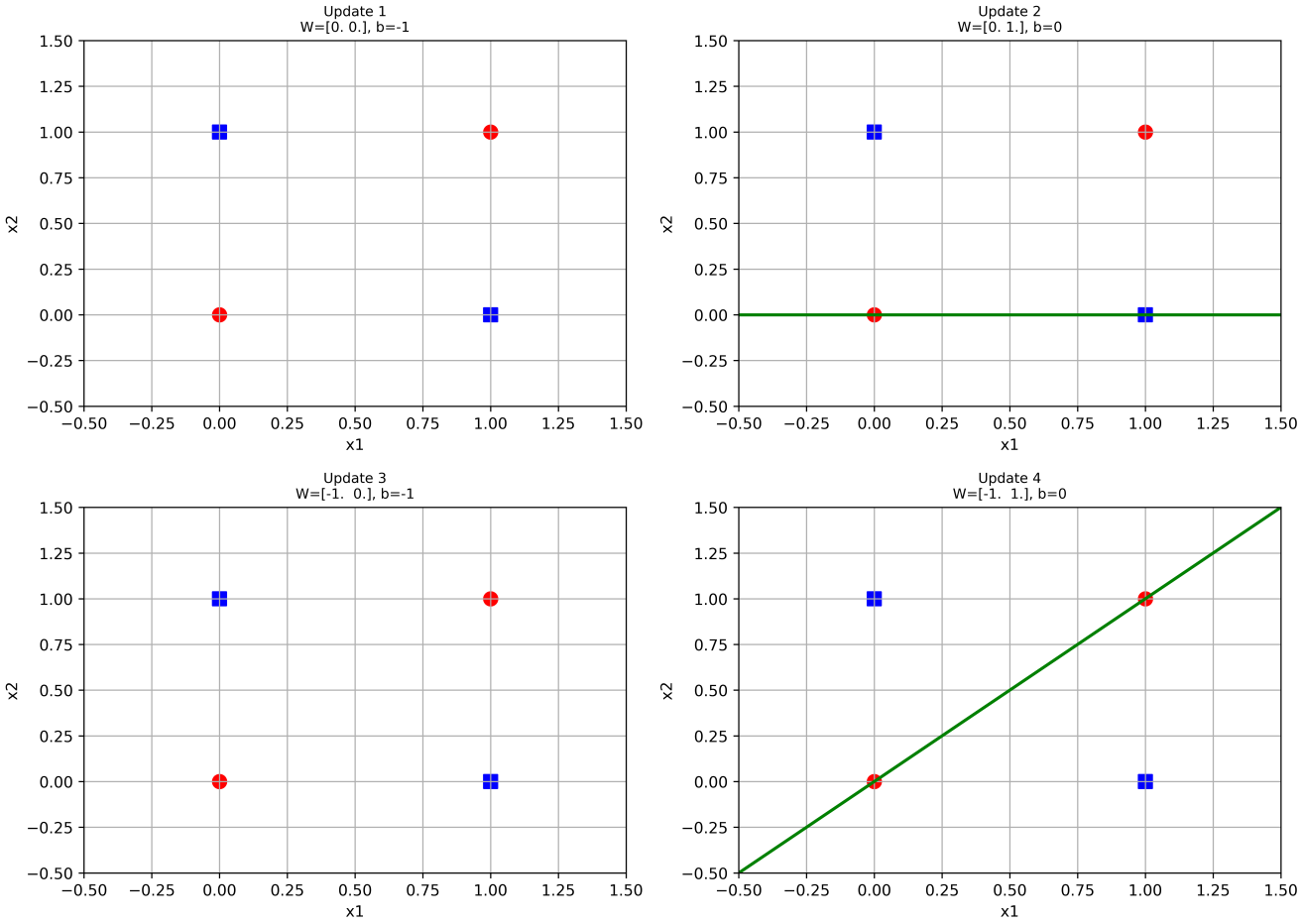
5. **Compare Grid Search and Randomized Search.**
   - **Grid Search** evaluates every possible combination of hyperparameters, whereas **Randomized Search** evaluates only a randomly selected subset of combinations.
   - Grid Search is computationally expensive and time-consuming, while Randomized Search is faster and more computationally efficient.
   - Grid Search guarantees finding the best combination within the specified search space, whereas Randomized Search finds a near-optimal solution with significantly less computation.
   - Grid Search is suitable for small search spaces, while Randomized Search is preferred for large search spaces with many hyperparameters.

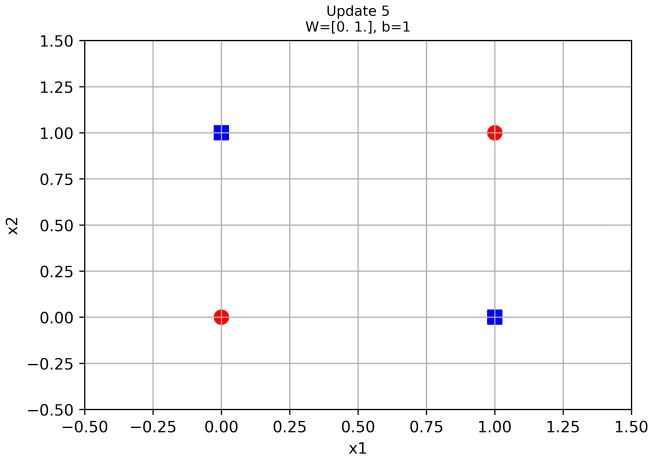
6. **Recommend the best MLP configuration.**
   The recommended MLP configuration consists of one hidden layer with 128 neurons, ReLU activation function, Adam optimizer, learning rate of 0.001, batch size of 16, training for 10 epochs, and a dropout rate of 0.0. This configuration provided the best balance between classification accuracy, computational efficiency, and generalization on the Fashion-MNIST dataset.

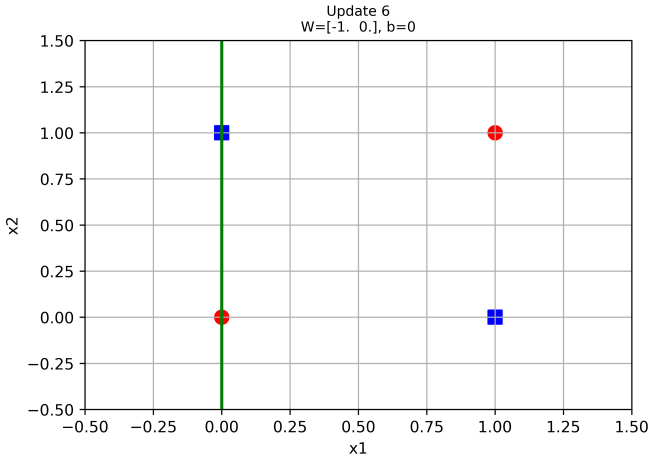
12. Perceptron Learning for XOR Gate

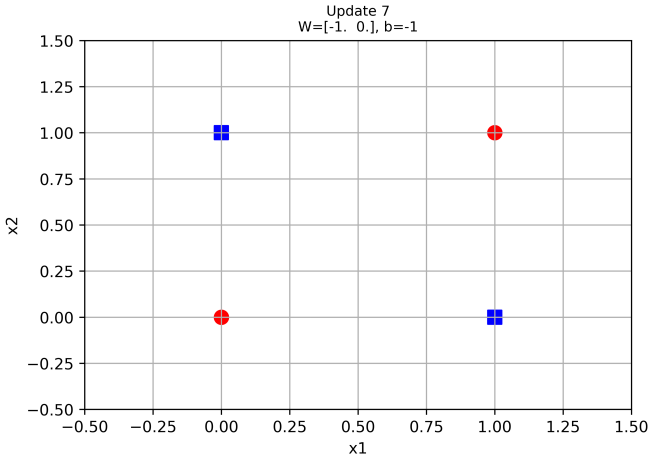
The XOR gate was implemented using the perceptron learning algorithm with initial weights and bias set to zero. During training, the weights and bias were updated after every misclassified sample, and the decision boundary corresponding to each update was plotted. Unlike the OR, AND, and NOT gates, the perceptron was unable to converge because the XOR dataset is not linearly separable.

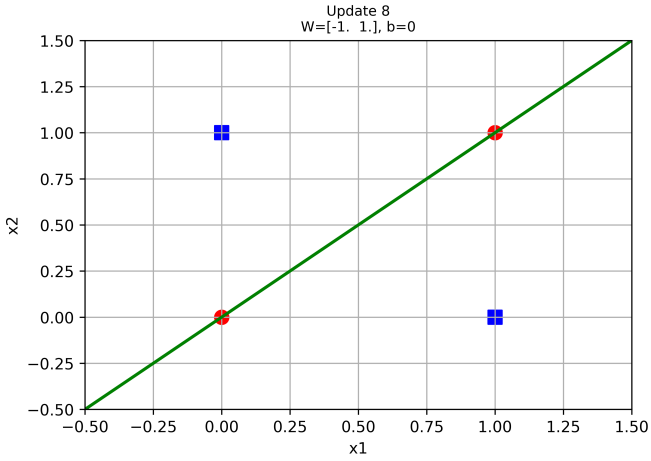
12.1 Learning Process

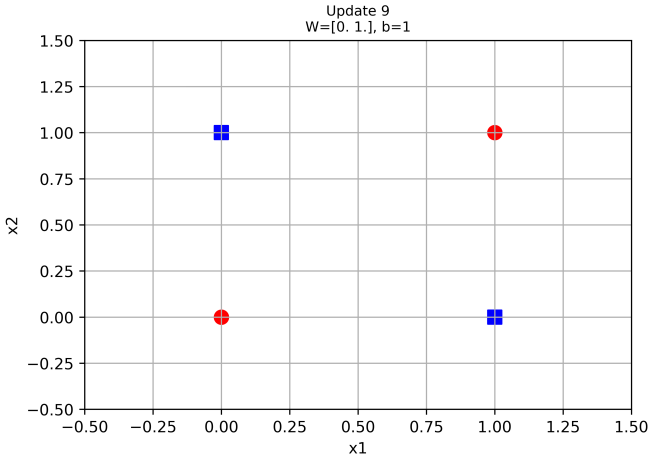


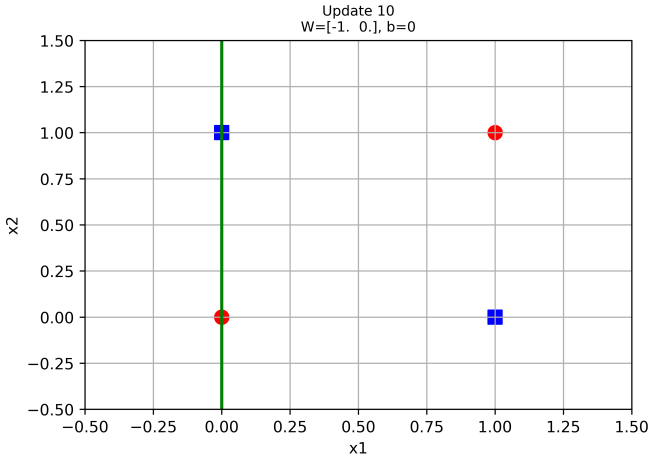

Update 5
W=[0. 1.], b=1
x1
x2

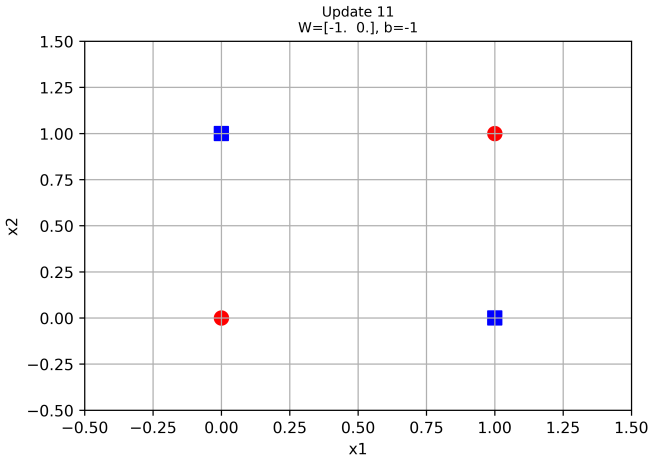

Update 6
W=[-1. 0.], b=0
x1
x2

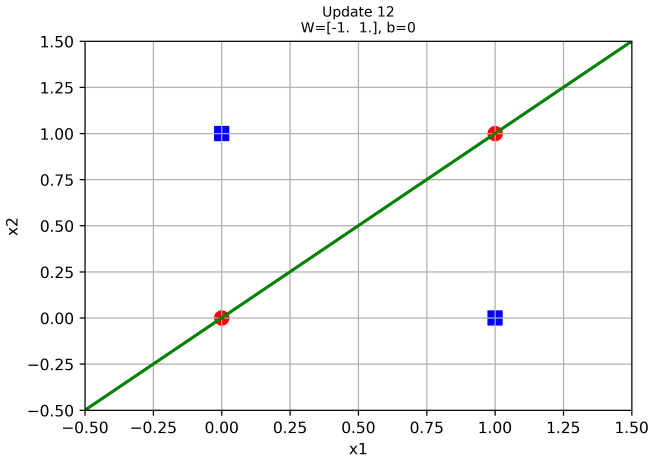

Update 7
W=[-1. 0.], b=-1
x1
x2

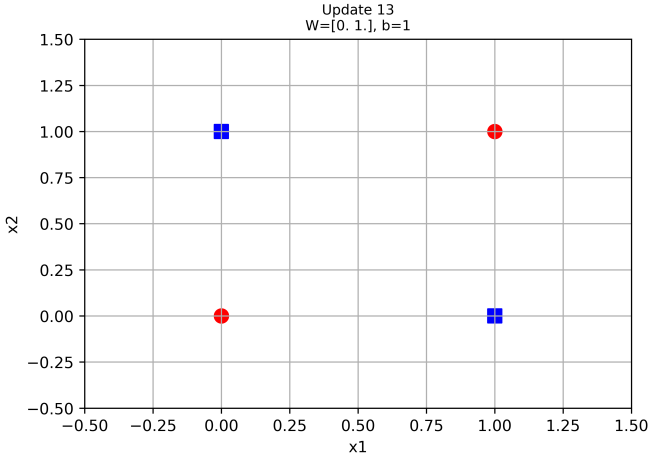

Update 8
W=[-1. 1.], b=0
x1
x2

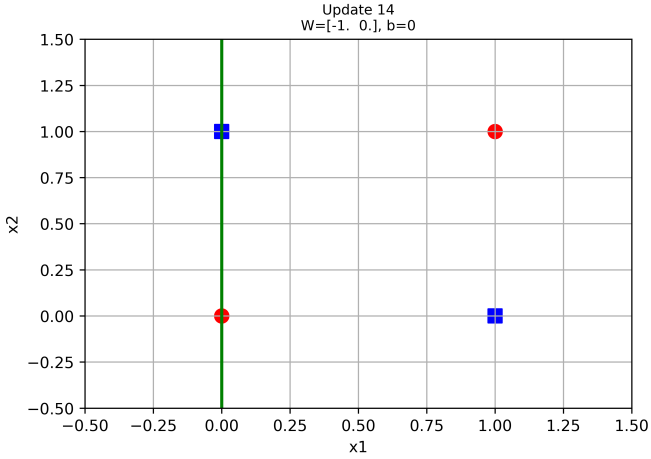

Update 9
W=[0. 1.], b=1
x1
x2

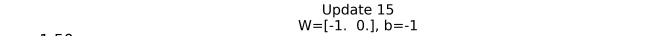

Update 10
W=[-1. 0.], b=0
x1
x2

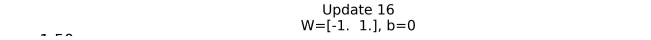

Update 11
W=[-1. 0.], b=-1
x1
x2

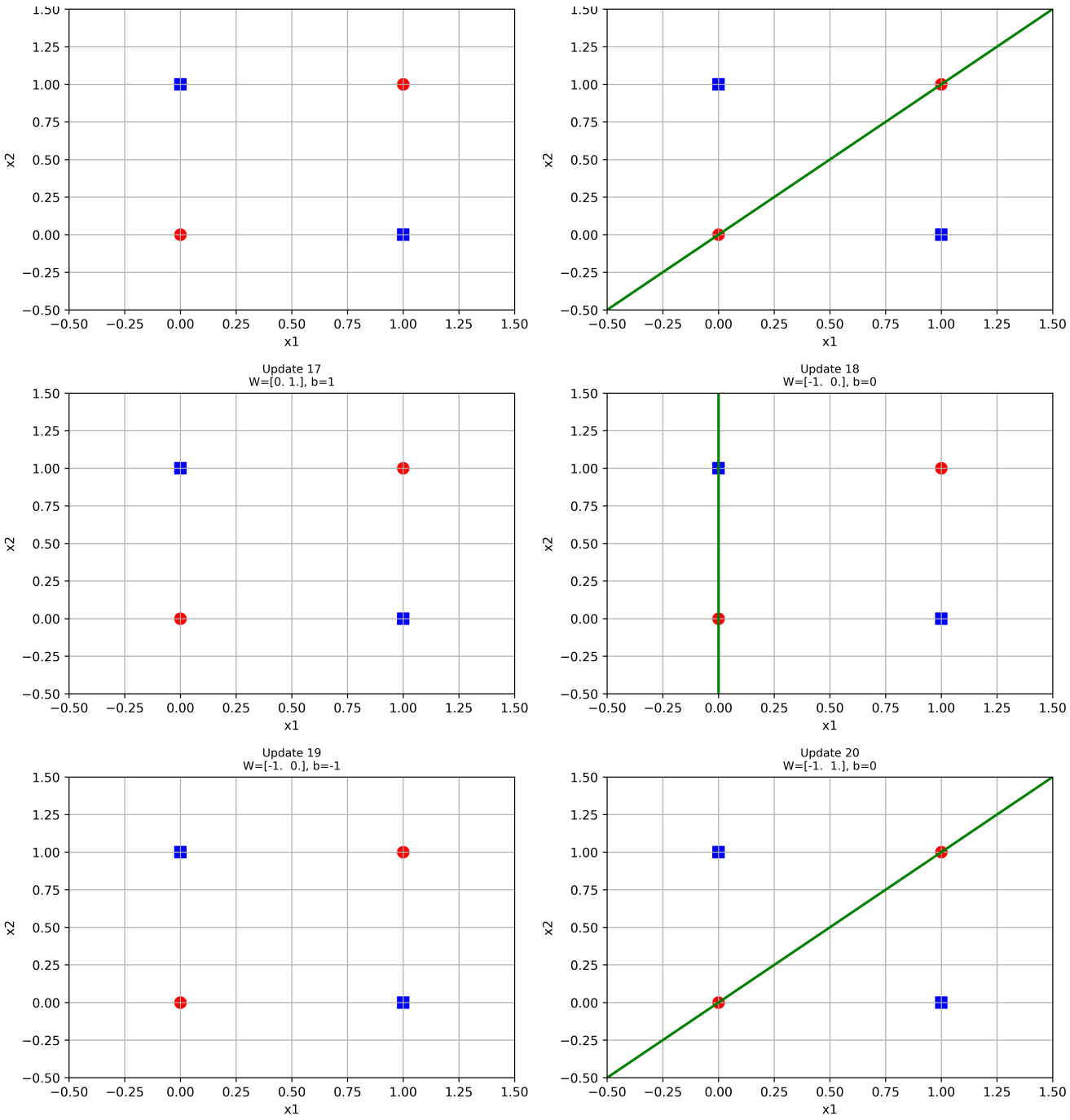

Update 12
W=[-1. 1.], b=0
x1
x2


Update 13
W=[0. 1.], b=1
x1
x2


Update 14
W=[-1. 0.], b=0
x1
x2


Update 15
W=[-1. 0.], b=-1


Update 16
W=[-1. 1.], b=0



12.2 Analysis

Inference:

- The decision boundary keeps changing throughout training and does not stabilize, indicating that the perceptron fails to find a single linear separator for the XOR dataset.
- The weight values oscillate between different configurations because correcting one misclassified sample causes another previously correct sample to become misclassified.

0.1 Pattern Preventing Convergence

The XOR gate consists of the following input-output pairs:

The positive samples $(0, 1)$ and $(1, 0)$ lie diagonally opposite each other, while the negative samples $(0, 0)$ and $(1, 1)$ occupy the remaining two corners of the input space. As a result, no single straight line can separate the two classes.

Consequently, the perceptron repeatedly updates its weights without reaching a stable solution, resulting in non-convergence. This demonstrates the limitation of a single-layer perceptron and motivates the use of multilayer perceptrons with hidden layers for solving non-linearly separable problems such as XOR.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.